

Отчёт об исследовательской работе по теме «AI for Automatic Algorithm Synthesis»

Студент: Игорь Ельчинов (БПМИ 251)

Научный руководитель: Рамон Антонио Родригес Залепинос

Москва, 2025

Содержание

1	Введение	3
2	Обзор литературы	4
2.1	AutoML: A Survey of the State-of-the-Art	4
2.2	The evolved transformer	10
2.3	AutoML-Zero: Evolving Machine Learning Algorithms From Scratch	12
2.4	MOAZ: A Multi-Objective AutoML-Zero Framework	13
2.5	Alphaevolve: A coding agent for scientific and algorithmic discovery	15
2.6	ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution	16
2.7	Automated design of agentic systems	18
2.8	Ale-bench: A benchmark for long-horizon objective-driven algorithm engineering	20
3	Доступные датасеты и бенчмарки	24
4	Постановка задачи и дальнейший план работы	24
4.1	Целевая функция. Бенчмарк СОСО	25
4.2	Пространство поиска. Библиотека Opytimizer	26
4.3	Общая схема работы агента	26

1 Введение

Автоматическое машинное обучение — это раздел машинного обучения, который ставит своей целью создание алгоритмов для решения конкретной задачи или их частей с как можно меньшим участием человека.

В 2010х годах глубокое обучение позволило достичь больших успехов в таких областях, как классификация изображений, поиск объектов на изображении и моделирование языка. Несмотря на все достижения, существенным недостатком новых моделей является ручной процесс их разработки методом проб и ошибок, что означает, что даже экспертам требуется много времени и ресурсов на разработку качественных моделей. Чтобы сократить издержки была предложена идея автоматизировать весь процесс разработки моделей (автоматическое машинное обучение, *automated machine learning*, AutoML). AutoML позволяет снизить нагрузку на дата-саентистов и применять МО на практике, не имея больших знаний статистики или машинного обучения.

После того как в статье [11] было показано, что автоматически можно генерировать алгоритмы, не уступающие аналогам, сделанным людьми, начался взрывной рост исследований в области поиска нейросетевых архитектур (Neural Architecture Search, NAS). Суть NAS заключается в поиске эффективных архитектур нейронных сетей за счёт комбинирования базовых операций, задающих пространство поиска.

Далее мы подробно рассмотрим несколько работ из области автоматического машинного обучения. Сначала мы рассмотрим обзорную статью 2021 года [10], которая перечисляет «классические» подходы в AutoML. Далее будет рассмотрен один из таких подходов, описанный в статье [3]. Затем будут рассмотрены две статьи ([1], [2]), подход которых заключается в генерации алгоритмов машинного обучения целиком «с чистого листа» с использованием программ на псевдокоде в качестве пространства поиска. После будут рассмотрены две статьи ([5], [6]), предлагающие использовать большие языковые модели для внесения правок в генерируемую программу и управления процессом эволюции. Мы завершим рассмотрением статьи [8], где предлагается использовать автоматическое машинное обучение для генерации сложных объектов, а именно целых агентов.

Затем будут рассмотрены основные датасеты и бенчмарки, используемые в AutoML, один из которых будет рассмотрен подробнее. В конце, на основе всех рассмотренных статей, будет поставлена задача исследования, а также сформулирован план дальнейшей работы.

2 Обзор литературы

2.1 AutoML: A Survey of the State-of-the-Art

Процесс разработки модели можно разделить на следующие этапы: подготовка данных, инженерия признаков (feature engineering), генерация модели и оценка качества её работы. Генерация модели может быть далее разделена на пространство поиска (т.е. принципы, на основе которых будет строиться модель) и методы оптимизации (оптимизация гиперпараметров и оптимизация архитектуры модели). Поиск нейросетевых архитектур (Neural Architecture Search, NAS) состоит из трёх главных составляющих: пространства поиска, методов оптимизации и методов оценки полученной архитектуры. В данной статье авторы рассматривают передовые методы в AutoML на всех этапах разработки модели.

Подготовка данных. Первым этапом является подготовка данных, которая включает в себя сбор, очистку и аугментацию.

Для некоторых задач, например, классификации изображений, есть хорошие открытые датасеты, однако для других задач данных может быть меньше, или работа с ними может быть затруднена из-за их персонального характера (например, задача классификации медицинских снимков). Поэтому вместо поиска данных в интернете некоторые исследователи предлагают их генерировать.

Для некоторых задач, особенно связанных с обучением с подкреплением (например, автономный транспорт), возможно использовать симуляторы. В открытом доступе есть такие инструменты, как OpenAI Gym (source?), которые позволяют исследователям сосредоточиться на разработке алгоритма, а не на поиске данных. Для других задач было предложено использовать генеративные состязательные сети для генерации синтетических данных.

Полученные данные содержат шумы, которые должны быть очищены. Было предложено несколько алгоритмов, например, BoostClean и AlphaClean, для автоматизации этого процесса. Однако их недостатком является применимость только в рамках конкретного датасета. Авторы статей [12] и [13] предлагают решения для очистки данных, которые могут использовать знания из предыдущих датасетов для обработки новых.

Аугментация данных служит регуляризатором, предотвращающим переобучение моделей. Для задач компьютерного зрения традиционно использовались простые трансформации (например, отзеркаливание), но некоторые исследователи предлагают использовать нейронные сети для аугментации изображений. Для текстовых данных используется замена слов на синонимичные или последовательный перевод текста между несколькими языками.

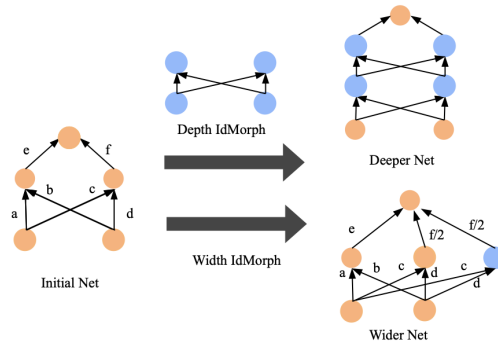


Рис. 1: Пример операций identity morphism transformations для morphism-based search space. [10]

Недостатками этих подходов является зависимость от человеческого знания. Чтобы исправить эту проблему, был предложен алгоритм AutoAugment, однако он требует большого числа вычислительных ресурсов.

Инженерия признаков (Feature Engineering). Данный этап обычно делится на отбор признаков (feature selection), построение признаков (feature construction) и извлечение признаков (feature extraction).

Отбор признаков используется для построения меньшего набора признаков за счёт отбора действительно существенных признаков. Основными методами поиска здесь являются полный перебор, случайный перебор и перебор на основе эвристик. Наиболее частыми методами случайного поиска являются симуляция отжига и эволюционные алгоритмы.

Построение признаков — это процесс, который создаёт новые признаки на основе комбинаций и преобразований старых. Так как перебрать все варианты вручную невозможно, были предложены алгоритмы, которые позволяют автоматизировать этот процесс. Самыми частыми методами являются решающие деревья, генетические алгоритмы и подходы, основанные на примечаниях. Последние обладают преимуществом, так как не требуют заранее определённого пространства преобразований.

Извлечение признаков позволяет уменьшить количество признаков. В отличие от отбора признаков, здесь исходные признаки также изменяются. Наиболее эффективными подходами в этой области являются PCA и LDA. Недавно также приобрёл популярность подход, который обучает нейронную сеть и использует состояния её внутренних слоёв как новые признаки.

Генерация модели: пространство поиска.

Пространство поиска задаёт принципы, на основе которых строится архитектура модели. Зачастую методы NAS строят нейросеть как комбинацию простых блоков, созданных людьми (например, разные виды свёрток). Ниже

будут рассмотрены основные виды пространства поиска.

Полное пространство поиска. Хотя данный подход накладывает наименьшие ограничения на архитектуру модели и прост в реализации, его существенным недостатком является большие вычислительные затраты.

Ячеистое пространство поиска (Sell-based search space). Данный подход основывается на наблюдении, что большинство успешных нейронных сетей используют некоторые блоки, которые затем последовательно объединяются в итоговую сеть. Так как теперь задача сводится к поиску оптимальной архитектуры одного блока, а не всей сети, этот подход требует значительно меньше вычислительных ресурсов. Однако и у этого метода есть свои недостатки, например, для обучения и оценки модели зачастую используется сеть с разным числом блоков, что может исказить результаты. Такие методы, как progressive-DARTS решают эту проблему за счёт большего числа итераций обучения с постепенным увеличением числа блоков. Эксперименты показывают, что этот метод позволяет генерировать нейронные сети с 2.50% ошибочных предсказаний на датасете CIFAR-10.

Иерархическое пространство поиска. Ещё один недостаток предыдущего подхода заключается в том, что он концентрируется на архитектуре отдельного блока, игнорируя архитектуру всей сети. Такие параметры, как количество блоков, всё ещё должны настраиваться вручную. Чтобы решить эту проблему, был предложен метод HierNAS, где блок более высокого уровня собирается как комбинация блоков предыдущего уровня.

Морфическое пространство (Morphism-based Search Space). Данный подход основывается на идее передачи знаний от одной модели к другой. Такие методы как дистилляция и transfer learning не позволяют перенести архитектуру модели, поэтому была предложена техника Net2Net. Он использует операции identity morphism transformations, чтобы строить на основе исходной модели идентичную, но с большим числом слоёв или с увеличенными слоями. Улучшенный метод network morphism [14] был предложен, чтобы исправить недостатки предыдущего подхода. Он позволяет дочерней нейронной сети унаследовать всё знание от родительской и продолжать развиваться дальше за значительно меньшее время.

Генерация модели: методы оптимизации. После того как было определено пространство поиска, необходимо выбрать метод, с помощью которого будет найдена оптимальная архитектура. Здесь будут рассмотрены основные подходы.

Эволюционные алгоритмы. Данный тип алгоритмов вдохновлён природной эволюцией. Для начала выбирается способ представления архитектуры нейронной сети, удобный для изменения и рекомбинации. Основными подхо-

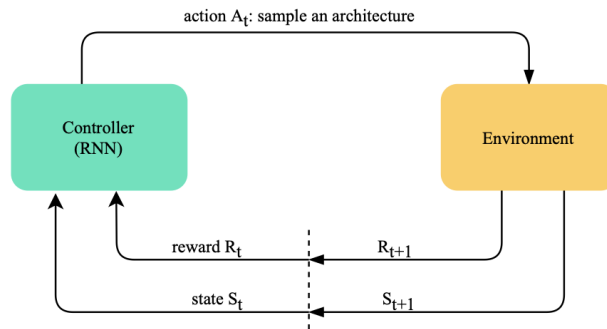


Рис. 2: Схема работы обучения с подкреплением для генерации архитектуры нейронной сети. [10]

дами являются DAG, GDP и Cellular encoding. Большинство эволюционных алгоритмов состоит из 4 шагов:

- **Отбор.** На этом этапе из всех ранее найденных архитектур отсеиваются слабые, чтобы остальные можно было использовать для стадии скрещивания. Основными методами являются отбор по приспособленности (fitness selection), отбор по рангу (rank selection) (для отбора кандидатов используется абсолютная и относительная точность соответственно) и турнирный отбор (tournament selection).
- **Скрещивание.** На этом этапе каждая пара выбранных кандидатов формирует потомство, наследующее половину генетической информации от каждого из родителей. Конкретный метод рекомбинации зависит от выбранного способа представления архитектуры нейронной сети.
- **Мутация.** После рекомбинации в полученную архитектуру вносятся случайные изменения, которые могут приводить как к потере работоспособности архитектуры, так и к поиску новых подходов.
- **Обновление популяции.** Так как в процессе алгоритма генерируется большое количество новых архитектур, популяцию необходимо обновлять, удаляя старые архитектуры по некоторому правилу.

Обучение с подкреплением. Впервые использование обучения с подкреплением для генерации архитектур было предложено в статье [15]. Здесь контроллер (рекуррентная нейронная сеть) в каждый момент времени выполняет действие, порождающее новую архитектуру, и получает в ответ информацию о текущем состоянии среды (процесс обучения выбранной архитектуры) и функцию награды (точность полученной модели). На основе этой работы

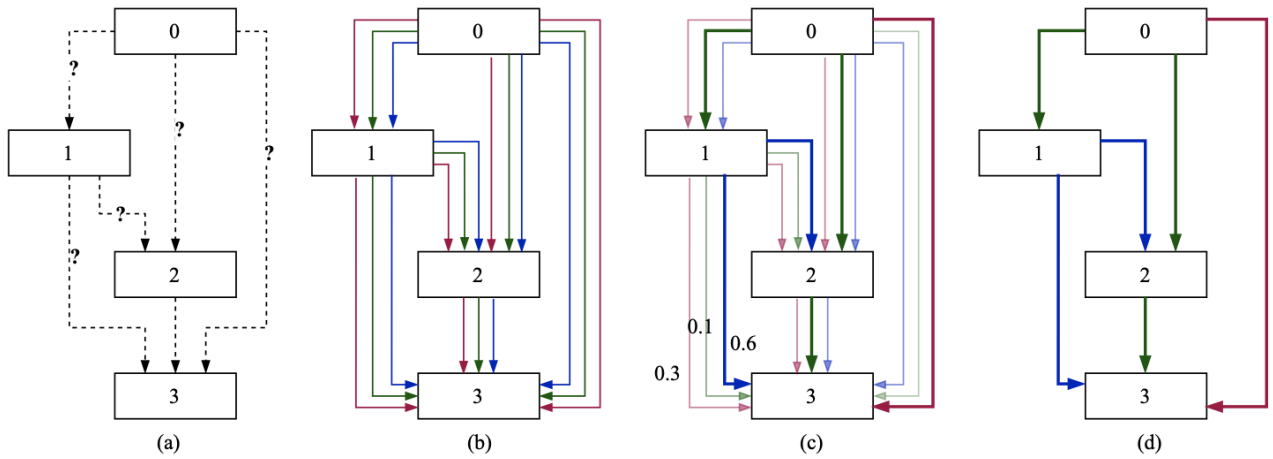


Рис. 3: Схема работы DARTS. Изначально операции на рёбрах неизвестны (a). До начала обучения операция на каждом ребре — это среднее из всех операций-кандидатов (b). Веса операций-кандидатов обновляются в процессе обучения (c). В финальной архитектуре используется операция с наибольшим весом (d). [10]

было предложено множество схожих подходов. Хотя обучение с подкреплением позволяет добиться выдающихся результатов в плане точности получаемой модели, оно слишком ресурсозатратно. Чтобы справиться с этой проблемой были предложены новые алгоритмы, например, BlockQNN и ENAS, которые используют асинхронное обучение и разделение параметров между всеми полученными архитектурами, чтобы ускорить генерацию модели.

Градиентный спуск. В отличие от предыдущих подходов, использующих дискретное пространство поиска, алгоритм DARTS использует градиентный спуск для поиска архитектуры в непрерывном дифференцируемом пространстве поиска. Здесь связь между узлами сети осуществляется с помощью взвешенной суммы всех операций-кандидатов. В процессе обучения веса операций меняются. Финальная архитектура задаётся операциями с наибольшим весом на момент окончания поиска. Недостатком такого подхода является большая вычислительная сложность, что особенно важно, линейная зависимость сложности от числа операций-кандидатов. Дальнейшие алгоритмы (GDAS, ProxylessNAS) предложили способы сократить количество вычислений. Ещё одной проблемой является вероятность того, что операции-кандидаты будут соперничать друг с другом. Эта проблема была решена в алгоритме DARTS+ с помощью остановки обучения, если нежелательные операции (например, skip-connection) начинают доминировать в сети.

Оптимизация на основе суррогатной модели (Surrogate Model-based Optimization). Идея подхода SMBO заключается в том, чтобы построить суррогатную мо-

дель, которая будет учитывать результаты оценки качества предыдущих архитектур и на основе их предсказывать наиболее многообещающую архитектуру. Таким образом, эти методы могут существенно сократить время поиска и повысить эффективность. В качестве суррогатной модели часто используются методы баесовской оптимизации, нейронные сети и ансамбли.

Случайный поиск и перебор по сетке. Исследования показывают, что случайный поиск является неплохим бейзлайном, который, при использовании некоторых оптимизационных трюков может быть почти настолько же эффективным, как методы на основе обучения с подкреплением и другие подходы.

Гибридные методы. Так как каждый из вышеперечисленных подходов имеет свои достоинства и недостатки, некоторые исследователи предложили комбинировать их.

Генерация модели: оптимизация гиперпараметров. Большая часть подходов использует одинаковый набор гиперпараметров для всех полученных моделей. Это означает, что после того как лучшая архитектура будет найдена, необходимо вручную настроить её гиперпараметры. Чтобы автоматизировать этот процесс, было предложено использовать такие подходы, как случайный поиск и перебор по сетке, методы баесовской оптимизации, а также методы на основе градиентного спуска.

Оценка модели. После того как очередная архитектура была получена, модель необходимо обучить и оценить её качество, что требует больших вычислительных затрат. Было предложено несколько методов оптимизации этого этапа.

Низкая точность (Low fidelity). Наиболее простым подходом является обучение модели лишь на части исходного датасета с уменьшенной размерностью (например, с меньшим разрешением в случае изображений). Также можно обучать упрощённую версию модели или использовать меньшее количество итераций обучения.

Разделение весов (Weight sharing). Использование одних и тех же весов для нескольких архитектур позволяет значительно ускорить процесс оценки модели.

Суррогатные методы (Surrogate). Использование суррогатной модели позволяет ускорить обучение и оценку полученных архитектур.

Преждевременная остановка (Early stopping). Изначально преждевременная остановка обучения использовалась в классическом МО для предотвращения переобучения; однако некоторые исследования показали, что её можно использовать для ускорения этапа оценки полученной архитектуры, заканчивая обучение модели, если для неё была предсказана недостаточная точность.

Открытые проблемы и дальнейшие направления для исследований. На данный момент у методов NAS есть ряд нерешённых проблем. Например, все описанные выше подходы используют созданные людьми операции в качестве базовых, что вносит некоторую предвзятость в получаемую архитектуру. Авторы статьи [1] предложили использовать в качестве базовых операций простые математические преобразования. Их подход способен генерировать сложные архитектуры с нуля.

Ещё одной задачей является применение AutoML в большем числе сфер. Сейчас исследования в основном сконцентрированы на задаче классификации изображений и лишь некоторые исследования пытаются применить данный подход к другим задачам.

Получаемые архитектуры зачастую сложно интерпретируемы и не обоснованы математически. Улучшение интерпретируемости AutoML является ещё одним направлением для исследований.

Также большой проблемой является воспроизводимость результатов. Было предложено несколько датасетов для сравнения различных методов друг с другом (NAS-Bench-101, NAS-Bench-201, NAS-Bench-NLP), однако здесь всё ещё есть пространство для дальнейших исследований.

Недостатком многих методов является низкая производительность. Кроме того, большинство из них требуют для обучения хорошо подготовленные размеченные данные без шумов.

Многие исследования рассматривали поиск архитектуры и оптимизацию гиперпараметров как отдельные задачи, однако они пересекаются по используемым методам, поэтому их можно осуществлять одновременно.

Главная цель AutoML — полностью автономный процесс разработки модели — всё ещё не достигнута, хотя такие инструменты, как TPOT, AutoWEAK, Auto-Sklearn, Auto-Keras и NNI, позволяют автоматизировать некоторые его части.

На данный момент большинство исследований концентрировались на решении конкретной задачи. Ещё одним направлением для исследований является разработка таких систем, которые могли бы легко дообучаться на новых данных, не забывая при этом старые знания.

2.2 The evolved transformer

В данной статье авторы предлагают применить эволюционный поиск алгоритмов к задачам, связанным с обработкой языка. Чтобы ускорить вычисления был предложен новый метод последовательных динамических препятствий (Progressive Dynamic Hurdles). В итоге была получена новая архи-

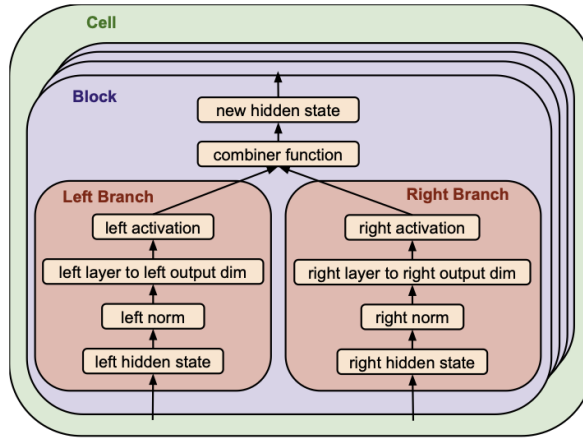


Рис. 4: Пространство поиска «развитого трансформера». [3].

текстура «развитый трансформер» (evolved transformer), которая превосходит трансформеры, созданные людьми, в области перевода текста.

Исследователи используют ячеистое пространство поиска. Архитектура представляется в виде нескольких ячеек, каждая из которых состоит из 6 (в случае энкодера) или 8 (в случае декодера) независимых блоков. Блок принимает на вход скрытые представления от предыдущих ячеек, обрабатывает их независимо, а затем комбинирует. Такое пространство поиска получается довольно большим, к тому же на обучение языковой модели требуется большое количество вычислительных ресурсов и времени. Чтобы решить эту проблему, исследователи предлагают две оптимизационные техники. Во-первых, в качестве начальной популяции используется уже зарекомендовавшая себя архитектура трансформера.

Вторая техника была названа последовательными динамическими препятствиями (Progressive Dynamic Hurdles). Её суть заключается в том, что полное число шагов обучаются лишь наиболее успешные модели. Более конкретно, первые m дочерние модели обучаются s_0 шагов, после чего вычисляется их средняя точность, которая объявляется препятствием h_0 . Следующие m моделей обучаются s_0 шагов, а затем ещё s_1 шагов, если их точность после оказалась больше h_0 . Когда достаточное количество моделей обучилось на полном в течение $s_0 + s_1$ итераций, их средняя точность объявляется препятствием h_1 , и так далее.

В результате поиска была найдена архитектура развитого трансформера, которая превосходит своего предшественника в задачах машинного перевода текста. Сравнения производились на бенчмарках WMT 2014 English-German, WMT 2014 English-French (En-Fr), WMT 2014 English-Czech (En-Cs) и the 1

Billion Word Language Model Benchmark (LM1B).

2.3 AutoML-Zero: Evolving Machine Learning Algorithms From Scratch

В данной статье авторы предлагают новый подход к генерации алгоритмов машинного обучения. До этого исследователи концентрировались на отдельных фрагментах алгоритма, таких как архитектура сети или используемые аугментации, сильно сужая пространство поиска или искажая результаты в пользу алгоритмов, схожих с теми, что создаются людьми (human bias). В новом исследовании авторы предлагают генерировать алгоритм целиком, включая предварительную инициализацию, способ обучения и формирования предсказания. На примере нескольких задач было показано, что генетические алгоритмы способны решать задачу поиска оптимального алгоритма в большом пространстве поиска, не только переизобретая известные алгоритмы, но и улучшая их.

Алгоритмы представляются в виде программ, состоящих из трёх базовых функций: **Setup**, **Predict** и **Learn** (см. Рис.5). Внутри каждой функции разрешено использование только базовых математических операций над скалярами, векторами и матрицами. Чтобы исключить генерацию схожих алгоритмов, ускорив тем самым поиск лучшего из них, был разработан способ сравнения таких программ, не учитывающий конкретную реализацию алгоритма (function equivalence checking, FEC).

Поиск подходящего алгоритма производился с помощью генетического алгоритма. Для ускорения поиска были использованы два улучшения: вышеупомянутый FEC и метод последовательных препятствий [3]. Также использовался параллельный отбор нескольких популяций со случайной миграцией между ними.

В ходе проведения нескольких экспериментов на задачах, таких как классификация изображений датасета CIFAR-10, выяснилось, что данный генетический алгоритм значительно лучше случайного поиска и способен создавать алгоритмы, превосходящие базовые решения, такие как линейная регрессия или полносвязные нейронные сети. Также в ходе эволюции удалось автоматически получить такие техники, как градиентный спуск и метод обратного распространения ошибки. Было показано, что при добавлении некоторых ограничений (малый объём данных или малое количество итераций обучения) генерируемые алгоритмы способны создавать адаптации к этим ограничениям.

Таким образом, авторы предложили новый подход к Auto ML и способ

```

# sX/vX/mX = scalar/vector/matrix at address X.
# "gaussian" produces Gaussian IID random numbers.
def Setup():
    # Initialize variables.
    m1 = gaussian(-1e-10, 9e-09) # 1st layer weights
    s3 = 4.1 # Set learning rate
    v4 = gaussian(-0.033, 0.01) # 2nd layer weights
def Predict(): # v0=features
    v6 = dot(m1, v0) # Apply 1st layer weights
    v7 = maximum(0, v6) # Apply ReLU
    s1 = dot(v7, v4) # Compute prediction
def Learn(): # s0=label
    v3 = heaviside(v6, 1.0) # ReLU gradient
    s1 = s0 - s1 # Compute error
    s2 = s1 * s3 # Scale by learning rate
    v2 = s2 * v3 # Approx. 2nd layer weight delta
    v3 = v2 * v4 # Gradient w.r.t. activations
    m0 = outer(v3, v0) # 1st layer weight delta
    m1 = m1 + m0 # Update 1st layer weights
    v4 = v2 + v4 # Update 2nd layer weights

```

Рис. 5: Пример кода, полученного алгоритмом AutoML-Zero. [1]

его реализации. В качестве дальнейшего направления исследования авторы предлагают улучшение алгоритма поиска алгоритмов. Также авторы отмечают, что в их подходе имеются значительные недостатки: в текущей реализации появление таких алгоритмов, как свёрточные нейронные сети или batch normalisation, невозможно ввиду архитектурных ограничений. Исправление этих недостатков также оставляется для будущих исследований.

2.4 MOAZ: A Multi-Objective AutoML-Zero Framework

Авторы данной статьи предлагают улучшение идеи, предложенной в статье AutoML-Zero (AZ). В отличие от своего предшественника, MOAZ старается оптимизировать не только точность получаемых алгоритмов, но и их эффективность, что важно для большинства реальных применений. Использование двух целей также позволяет улучшить точность получаемых алгоритмов и упростить их, что значительно облегчает интерпретацию результата.

Для реализации концепции многоцелевой оптимизации авторы статьи предложили следующие изменения. Например, была изменена оптимизируемая формула: берётся минимум из сложности алгоритма и ошибки с услови-

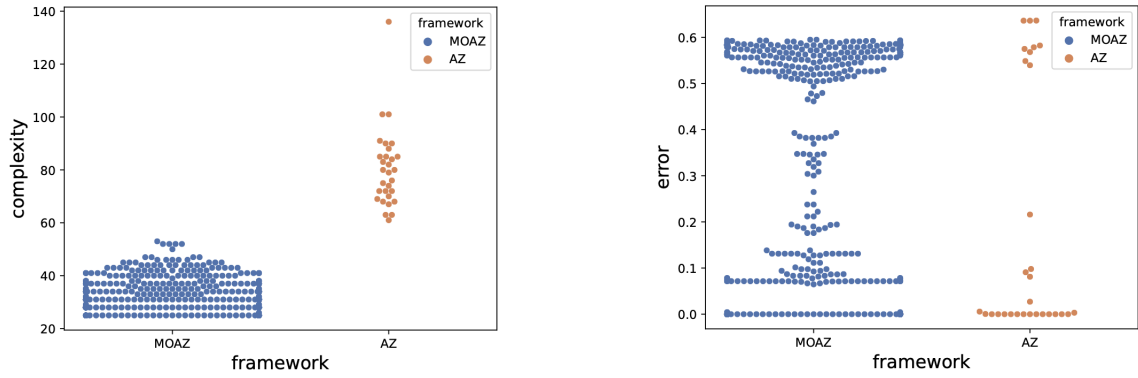


Рис. 6: Сравнение сложности и ошибки алгоритмов, получаемых методами AZ и MOAZ. [2]

ем, что оба показателя не превышают максимально допустимые значения (сложность алгоритма определяется как суммарное число FLOPs в функциях **Setup**, **Learn** и **Predict**). Для оптимизации такой функции используется модификация алгоритма NSGA-II с изменёнными функциями выбора доминирующего кандидата и кроссинговера.

В ходе запусков экспериментов было показано, что предложенный подход превосходит AZ не только в терминах сложности получаемых алгоритмов, но и по скорости и стабильности поиска достаточно хорошего решения (определяемого как решение с ошибкой меньше 10^{-1}). После тестирования на нескольких наборах искусственных задач было показано, что MOAZ находит достаточно хорошее решение для большего процента задач, причём количество запусков до первого удачного решения сильно меньше, чем у AZ.

Таким образом, предложенный в данной статье подход позволил получать алгоритмы сопоставимого качества (см. Рис. 6) быстрее и стабильнее, чем подход AZ. Полученные алгоритмы также эффективнее и легче поддаются интерпретации, что уменьшает нагрузку на человека на этапе постобработки полученного решения. Авторы статьи предлагают пути дальнейшего улучшения предложенного подхода путём добавления новых целей.

2.5 Alphaevolve: A coding agent for scientific and algorithmic discovery

В данной статье авторы предлагают агента для написания программ на основе LLM и эволюционного поиска. AlphaEvolve способен решать широкий спектр научных и инженерных задач, где решение может быть запущено и оценено автоматически.

Процесс поиска решения является эволюционным алгоритмом и состоит из последовательных запусков следующего цикла:

1. Родительские программы извлекаются из базы данных
2. На основе предложенных ранее решений и общего контекста задачи генерируется запрос к LLM
3. LLM предлагает свои правки к существующему решению
4. Полученные решения автоматически оцениваются
5. Наиболее успешные кандидаты записываются обратно в базу данных

AlphaEvolve может работать с целыми кодовыми базами. Конкретные участки, доступные для изменения, помечаются с помощью комментариев, что позволяет использовать агента, внося лишь незначительные правки в исходную кодовую базу. Также от пользователя требуется функция `evaluate()`, которая запускает решение и вычисляет значения метрик, которые необходимо оптимизировать.

В ходе тестирования AlphaEvolve был запущен на множестве разнообразных задач: от открытых математических задач (быстрое перемножение матриц, проблема минимального перекрытия Эрдеша и т.п.) до практических задач по оптимизации вычислительной экосистемы компании Google (улучшенный алгоритм построения расписаний для дата-центров, оптимизация ядер нейросети Gemini и т. д.). Все эти задачи показали, что предложенный агент способен находить решения сложных задач и значительно улучшает возможности отдельных LLM.

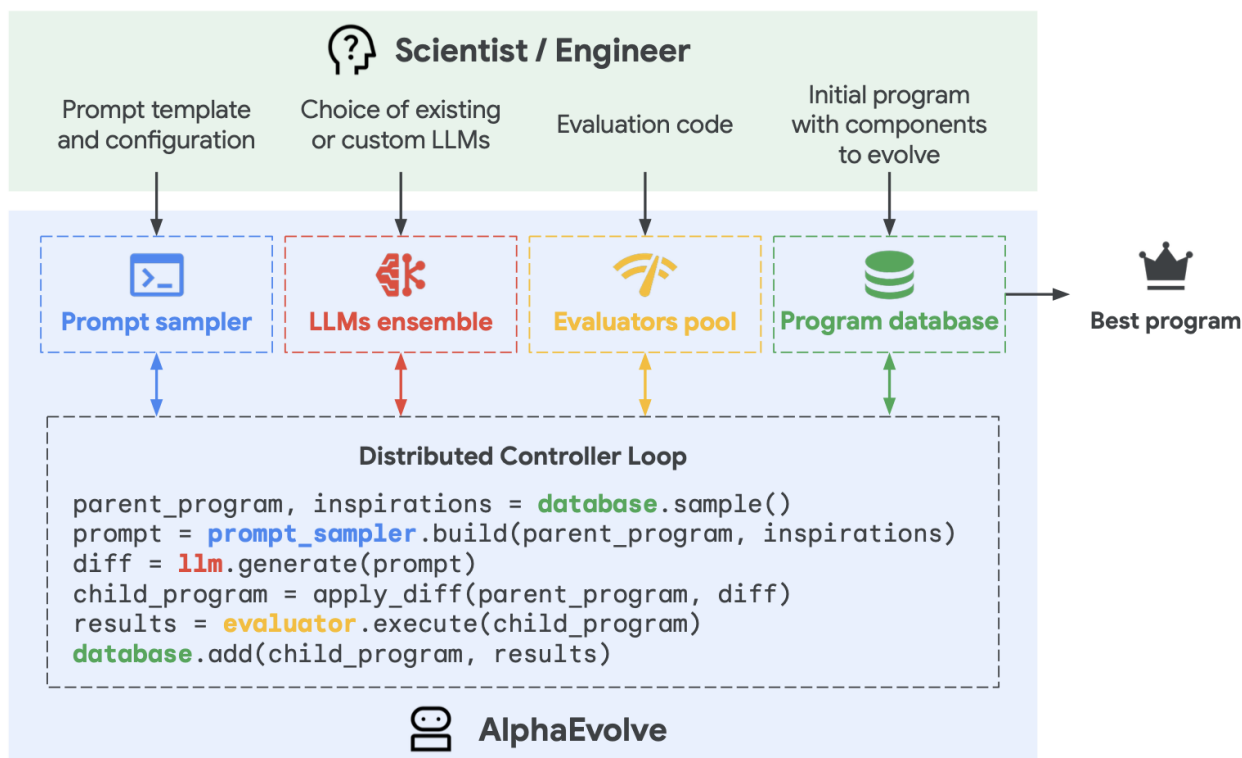


Рис. 7: Схема работы AlphaEvolve. [6].

2.6 ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution

В данной статье предлагается новый подход к AutoML, основанный на применении LLM. Предложенный подход значительно эффективнее существовавших решений на базе LLM. Открытый исходный код открывает возможности для дальнейших экспериментов.

ShinkaEvolve предлагает три основные архитектурные улучшения:

1. Смешанный выбор новых кандидатов (из родительской популяции и из архива лучших представителей данной популяции). Исследователи подчёркивают важность баланса между использованием уже зарекомендовавших себя подходов и поиском новых.
2. Мутация кандидата осуществляется через предложение по улучшению от LLM. Недостаточно изменённые решения (используется сравнение эмбедингов и оценка от LLM) отбраковываются.
3. Изменённая процедура запуска и оценки качества нового решения

Более подробно подход ShinkaEvolve заключается в следующем:

Отбор кандидатов

В архиве хранится фиксированное число программ с их точностью работы и мета информацией. Во время мутации БЯМ доступна программа из родительской популяции, несколько лучших, а также несколько случайных программ из архива. Популяция делится на «острова», эволюционирующие по отдельности. Между островами возможна миграция; однако, чтобы поддерживать уникальность каждого из них, лучшее решение в каждой из популяций исключается из процесса миграции. Исследователи также подчёркивают важность баланса между использованием эффективных подходов и поиском новых. Вероятность выбора программы в качестве родительской зависит от её точности, а также от количества и медианной точности её потомков.

Мутация кандидатов и оценка новизны

Перед запуском процесса эволюции выбирается набор LLM, которые будут осуществлять мутации, а также их параметры (температура, ограничение на использование размышлений и т.п.). Представлены три вида мутаций:

1. Изменение отдельного блока программы
2. Переписать программу с нуля
3. Скрещивание программы с дополнительной программой из архива

Оценка новизны производится следующим образом: сначала вычисляются эмбединги программ, которые затем сравниваются. Если похожесть двух программ превышает порог, то код отправляется LLM, которая оценивает, имеются ли существенные различия.

Запуск и оценка качества нового решения

После запуска каждого решения оно записывается в архив. К нему также добавляется информация о точности решения, а также набор заранее выбранных метрик вместе с текстовым отзывом о решении. Так как эффективность разных LLM может отличаться, для каждой LLM оценивается её вклад в улучшение качества решений (алгоритм UCB1); вероятность выбрать каждую модель в качестве мутатора зависит от этих параметров. ShinkaEvolve использует «блокнот» куда записываются описания всех решений, а также общие стратегии их оптимизации. Этого блокнот добавляется к каждому запросу на изменение программы.

Алгоритм ShinkaEvolve был протестирован на 4 задачах: упаковка кругов, AIME, ALE-Bench, задача обучения балансировщика для архитектуры MoE. На всех задачах алгоритм смог улучшить базовые решения. В одной из задач бенчмарка ALE-Bench ShinkaEvolve смог найти решение, занявшее второе место в рейтинге, а в последней задаче ShinkaEvolve предложил улучшение к

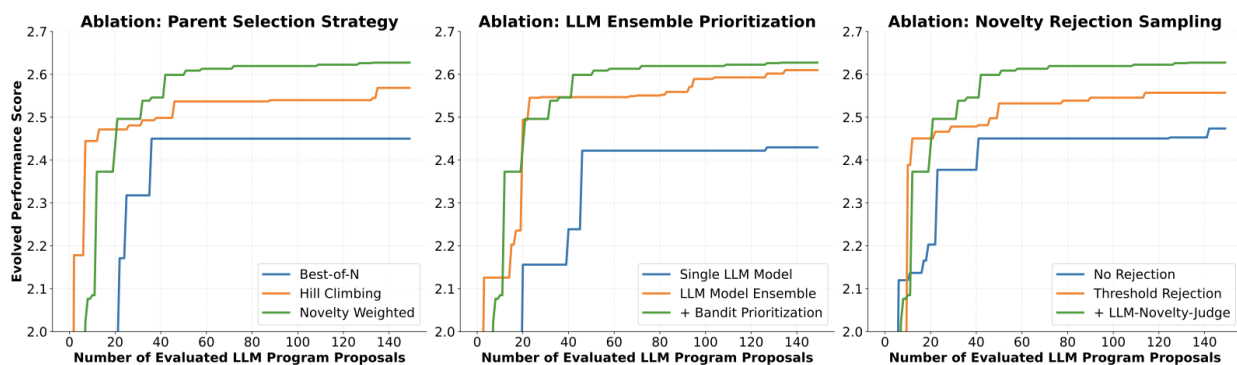


Рис. 8: Сравнение эффективности ShinkaEvolve без предложенных улучшений и с ними. [5]

популярной архитектуре балансировщика. Запуск на этих задачах показал, что почти все предложенные улучшения к алгоритму поиска решений дают значительное увеличение эффективности поиска. Исключение составляет только использование LLM для оценки новизны программ. Эксперименты показывают, что он даёт лишь незначительное улучшение по сравнению с эмбедингами, что говорит о том, что одних только их уже достаточно для отсеивания слишком похожих кандидатов.

Таким образом, авторы данной статьи предлагают новый подход, который оптимизирует использование LLM, позволяя значительно улучшить поиск решения. Авторы отмечают, что недостатками их подхода являются отсутствие возможности автоматической настройки баланса между использованием эффективных и поиском новых решений, а также ограниченный круг применимости из-за того, что подход заточен на цели в виде хорошо определённых числовых показателей. В качестве дальнейшего направления для исследований авторы видят устранение этих недостатков.

2.7 Automated design of agentic systems

Большие языковые модели часто используются в качестве агентов общего назначения. Несмотря на все успехи таких агентов, для многих задач требуется сложная система, имеющая доступ к внешним инструментам, таким как запуск кода или поисковая система. В результате было предложено много эффективных модулей для таких систем. Основываясь на опыте других областей машинного обучения, авторы данной статьи предлагают заменить ручную разработку отдельных модулей или даже агентов целиком их автоматической генерацией. Новая парадигма называется автоматическая разработка агентных систем (Automated Design of Agentic Systems, ADAS). Для решения

данной задачи был предложен алгоритм мета-агентного поиска (Meta Agent Search), который в ходе экспериментов смог сгенерировать агента, превосходящего созданные вручную аналоги в задачах понимания текста и решения математических задач.

Сначала авторы статьи определяют автоматическую разработку агентных систем как использование алгоритма поиска для нахождения агентных систем в пространстве поиска, которые оптимизируют целевую функцию. В данной работе в качестве пространства поиска используется пространство всех программ на некотором языке программирования. Такое пространство поиска позволяет найти любую возможную архитектуру агентной системы, а также предоставляет большую интерпретируемость по сравнению с другими пространствами.

Чтобы найти новые успешные архитектуры в таком большом пространстве поиска был предложен алгоритм мета-агентного поиска. Он состоит из следующих шагов:

1. Архив инициализируется базовыми решениями (опционально)
2. На основе архива генерируется новый агент: сначала высокоуровневое описание, а затем реализация в коде, после чего он проверяется на новизну
3. Далее агент оценивается на валидационной выборке, проходя несколько итераций улучшения при необходимости
4. Агент и информация о его работе на валидационной выборке добавляются в архив; процесс продолжается с обновлённым архивом, пока не будет достигнуто максимальное количество итераций

Исследователи провели эксперименты на бенчмарке Abstraction and Reasoning Corpus, четырёх бенчмарках для проверки способностей агентов в разных областях. Также была проверена возможность использовать агентов, созданных под математические задачи, для решения задач другого рода. Во всех экспериментах было обнаружено, что полученные агенты значительно превосходят аналоги созданные людьми, и сохраняют преимущество даже при использовании для других задач или с другой базовой моделью.

Таким образом, в данной статье предлагается новое направление для исследований, а именно автоматическая разработка агентных систем. Было предложено пространство поиска в виде пространства программ на выбранном языке программирования, а в качестве метода поиска — мета-агентный

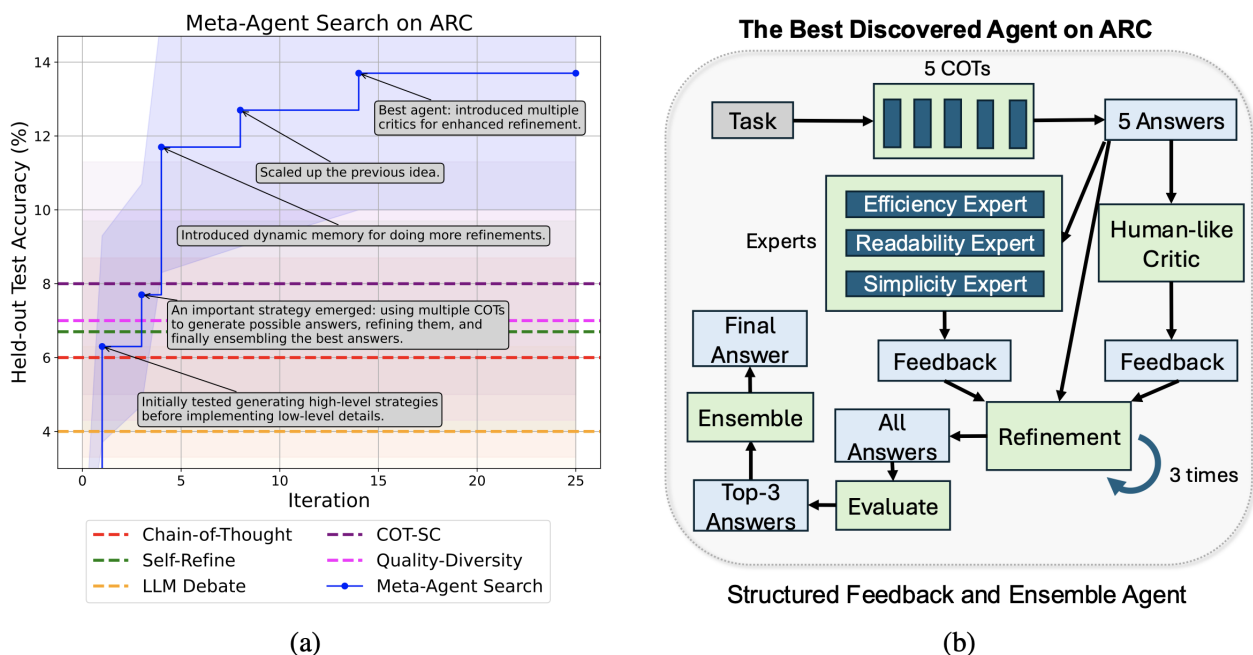


Рис. 9: Результат мета-агентного поиска для бенчмарка ARC: (a) мета-агентный поиск последовательно улучшает точность агента, (b) архитектура найденного агента. [8]

поиск. Авторы обозначают множество направлений для дальнейших исследований: от предложений по улучшению алгоритма поиска до идей по практическому применению, например, дообучение агентов на новых данных, собранных после их запуска (Online Continual Learning).

2.8 Ale-bench: A benchmark for long-horizon objective-driven algorithm engineering

Авторы данной статьи предлагают новый бенчмарк для оценки способности LLM генерировать решения алгоритмических задач. В отличие от предыдущих бенчмарков, использовавших задачи с заранее известным ответом, здесь используются более сложные задачи, а именно задачи на оптимизацию, точный оптимум которых недостижим. Это позволяет адекватно оценивать ИИ-системы, для которых задачи первого типа становятся слишком простыми.

Бенчмарк представляет из себя датасет из 10 задач в облегчённой или 40 задач в полной версии, взятых с туров AtCoder Heuristic Contest. К каждой задаче прилагается условие, программа-оценщик, визуализатор поведения отправленного решения и лидерборда. Так как бенчмарк создавался в

партнёрстве с AtCoder, все данные предоставлены официально, что гарантирует безопасное использование бенчмарка.

Бенчмарк реализован в качестве библиотеки на ЯП Python, цель которой состоит в том, чтобы позволить ИИ-системам решать задачи в формате, максимально приближенном к реальным АНС. Ключевую роль здесь играет объект **Session**, который запускает таймер конкурса и предоставляет ИИ-системе протокол взаимодействия с тестирующей системой. ИИ может выполнять любое из 4 действий:

1. Просматривать условие задачи
2. Запустить решение на открытых тестах, чтобы проверить решение
3. Визуализировать работу решения
4. Отправить финальный вариант решения

После выполнения действия 4 или окончания времени сессия завершается, и объект **Session** выдаёт оценку решения на закрытых тестах.

По результатам запусков ИИ-системы формируется несколько метрик. Первая группа вычисляется для каждой задачи отдельно:

1. Счёт в конкретной задаче
2. Ранг (rank) — место в рейтинге
3. Результативность (performance) — метрика, вычисляемая через счёт и ранг по алгоритму Elo-rating.

Вторая группа считается по всем задачам:

1. Средняя результативность по всем задачам
2. Рейтинг по системе, которая используется на настоящих АНС

Авторы отмечают, что рейтинг может необъективно оценивать модель, так как одно крайне успешное участие может завысить общий рейтинг ИИ-системы. К тому же использование средней результативности вместе с её распределением по задачам позволяет выявить, насколько широк спектр задач, в которых ИИ показывает достойные результаты.

Также в данной статье в качестве бэйзлайна была предложена система Ale-Agent. Ale-Agent использует две техники: запросы с общим знанием о задаче (в запросе к LLM сразу прилагаются стандартные идеи для решения задач такого типа) и ориентированный на разнообразие поиск решения.

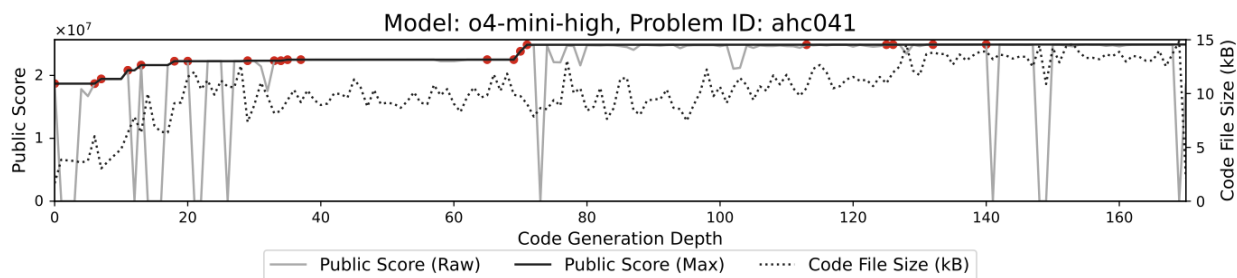


Рис. 10: Прогресс модели 04-high в решении конкретной задачи. [9].

На данном бенчмарке было проведено множество экспериментов с разными моделями и стратегиями их использования. В первом эксперименте моделям давали всего одну попытку. Самой результативной оказалась модель 03-high, будучи единственной моделью со средней результативностью более 1000. Ни одна из моделей не достигла результативности 2000 в какой-либо из задач, что подчёркивает сложность бенчмарка.

Во втором эксперименте использовалась стратегия итеративного улучшения. Четыре модели (o4-mini-high, GPT-4.1 mini, DeepSeek-R1, Gemini 2.5 Pro) в течение 4 часов последовательно улучшали свои решения. В итоге самой результативной моделью оказалась o4-mini-high (средняя производительность 1520 и). Модели o4-mini-high и GPT-4.1 mini были единственными, достигшими результативности хотя бы 400 на всех задачах, при этом последняя имеет меньший процент задач с результативностью хотя бы 2000 и меньшую среднюю результативность, чем DeepSeek-R1.

По результатам экспериментов рейтинг модели o4-mini-high сопоставим с лучшими 11.8% людей. Однако, если посмотреть на другие показатели, например, долю задач, где была достигнута результативность 1600 или 2400, то в среднем эти показатели значительно выше у людей. Это показывает, почему рейтинг не самый информативный показатель применительно к языковым моделям.

Ещё один эксперимент сравнивал стратегию последовательного улучшения со стратегией «параллельного поиска» (многократный запуск бенчмарка с одной попыткой на задачу). В результате стратегия последовательного улучшения приводила к значительно большей результативности. Это говорит о том, что стратегия многих попыток не приводит к хорошим результатам на бенчмарке Ale-Bench. Чтобы достичь большой результативности, нужно последовательно улучшать решения, основываясь на предоставляемой обратной связи.

По результатам экспериментов не было обнаружено связи между датой,

где заканчивались знания модели, и её результативностью на задачах. Также в решениях моделей не было найдено признаков плагиата.

Таким образом, в данной статье был предложен бенчмарк Ale-Bench, который проверяет способность ИИ-систем создавать и улучшать алгоритмы. Основным недостатком бенчмарка исследователи называют малое число задач. В качестве дальнейшего направления работы предлагается использование LLM для синтеза новых задач, что позволит уменьшить риск переобучения тестируемых ИИ-систем под конкретные типы и формулировки задач.

3 Доступные датасеты и бенчмарки

Так как целью AutoML было автоматизировать процесс разработки модели для решения конкретных задач, то для оценки используются датасеты для этой задачи. Изначально автоматическое машинное обучение применялось преимущественно к задаче классификации изображений, поэтому для оценки использовались стандартные для области компьютерного зрения датасеты, такие как MNIST, CIFAR-10, ImageNet и т.п. Затем подходы AutoML начали применять и к другим областям. Для оценки решения в области обработки языка могут использоваться WMT 2014 English-German, WMT 2014 English-French, WMT 2014 English-Czech или the 1 Billion Word Language Model Benchmark, а например, агентов оценивают на бенчмарках Abstraction and Reasoning Corpus, DROP, MGSM, MMLU и GPQA.

Однако использование таких датасетов создаёт трудности при сравнении двух систем, если для их оценки использовались разные датасеты. Также эксперименты могут различаться по используемым гиперпараметрам, вычислительным ресурсам и т.п. Чтобы исправить эту проблему были предложены специализированные датасеты, такие как NAS-Bench-101, NAS-Bench-201 для задачи классификации изображений и NAS-Bench-NLP для задач из области автоматической обработки языка.

Ещё одна группа бенчмарков позволяет сравнивать способности языковых моделей генерировать решения алгоритмических задач. Среди них такие бенчмарки, как CodeContests, LiveCodeBench и Ale-Bench. Более подробное описание последнего было представлено ранее.

4 Постановка задачи и дальнейший план работы

В рамках данной работы планируется создать агента для автоматической генерации оптимизатора заранее выбранной целевой функции. Основываясь на подходах, описанных в статьях [6] и [5], мы будем использовать генетический алгоритм с большими языковыми моделями в качестве источника мутаций, так как этот подход является одним из самых эффективных на данный момент. Чтобы сократить пространство поиска, мы будем использовать эвристики из библиотеки Opytimizer [17] в качестве элементов, из которых будет строиться оптимизатор. Будут использоваться целевые функции из бенчмарка СОСО [16].

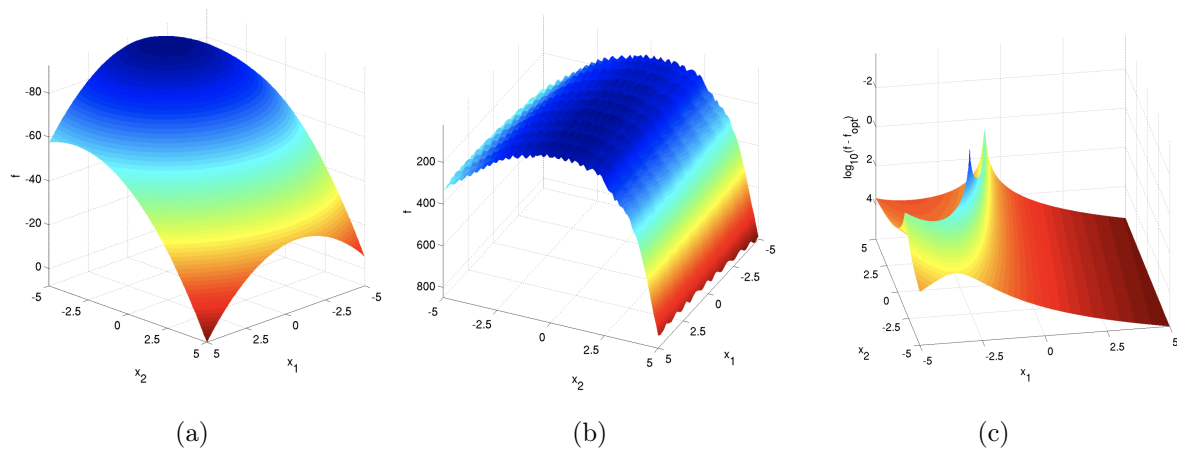


Рис. 11: Примеры функций для оптимизации из бенчмарка СОСО: (a) сферическая функция, (b) функция Растригина, (c) функция Розенброка. [18]

4.1 Целевая функция. Бенчмарк СОСО

Бенчмарк СОСО был создан для автоматизации трудоёмкой и повторяющейся работы по оценке качества работы оптимизаторов. Основными понятиями СОСО являются:

- Функция — класс отображений, устроенный схожим образом
- Инстанс — конкретная реализация функции
- Целевое значение — заранее оценённое как оптимальное значение индикатора качества
- Время работы — количество вызовов целевой функции перед достижением целевого значения
- Задача — совокупность конкретного инстанса, на котором запускается оптимизатор, и целевого значения
- Набор тестов — коллекция проблем

В рамках нашей работы целью будет *разработать метод автоматического синтеза оптимизатора, имеющего минимально возможное среднее ожидаемое время работы на заданном наборе тестов.*

СОСО предоставляет несколько наборов тестов, отличающихся по размерности, количеству целей, зашумлённости и т.д. Мы будем использовать самый стандартный набор `bbob`, который содержит 24 незашумлённые функции. Функции разделены на 5 групп по сложности их оптимизации. Примеры функций из этого набора представлены на Рис. 11.

4.2 Пространство поиска. Библиотека Opytimizer

Opytimizer — это библиотека с открытым кодом на Python для оптимизации целевых функций с помощью мета-эвристик. Opytimizer предоставляет почти 90 различных мета-эвристик 7 различных типов (эволюционные алгоритмы, алгоритмы на основе роевого интеллекта, на основе природных явлений и др.), предоставляющих единое API. Примеры алгоритмов из Opytimizer:

- Поиск гармонии (Harmony search) — эволюционный алгоритм, вдохновлённый процессом музыкальной импровизации.
- Алгоритм водного цикла (Water cycle algorithm) — алгоритм на основе имитации природных явлений, а именно процесса формирования рек.
- Алгоритм летучих мышей (Bat algorithm) — алгоритм роевого интеллекта, вдохновлённый эхолокацией летучих мышей.

В рамках данной работы библиотека будет выступать в качестве источника примитивов — наборов готовых алгоритмов, которые агент сможет комбинировать. Хотя набор функций большой, он всё же конечен, что обуславливает необходимость их комбинации для достижения лучших результатов на конкретных задачах. Стандартизированный API позволяет автоматически валидировать сгенерированный код, а возможности для визуализации дают обратную связь о поведении синтезированного алгоритма.

4.3 Общая схема работы агента

Основная задача агента — на основе описания целевой функции из бенчмарка СОСО и доступного каталога алгоритмов библиотеки Opytimizer, сгенерировать оптимизатор, который минимизирует ожидаемое время работы на заданном классе функций.

Для синтеза оптимизатора будут использоваться языковые модели, предоставляющие доступ по API, а также модели с открытым исходным кодом, которые можно развернуть локально. При выборе моделей будут учитываться такие показатели, как качество генерации кода (может оцениваться по бенчмаркам, например, HumanEval или MBPP), способность к исправлению ошибок, доступность (стоимость запроса для облачных моделей; наличие открытых весов, требования к видеокарте для локальных моделей). Конкретный список моделей будет сформирован позднее.

Список литературы

- [1] Esteban Real, Chen Liang, David So, and Quoc Le. AutoML-zero: Evolving machine learning algorithms from scratch. In Hal Daumé III and Aarti Singh, editors, Proceedings of the 37th International Conference on Machine Learning, volume 119 of Proceedings of Machine Learning Research, pages 8007–8019. PMLR, 13–18 Jul 2020.
- [2] Guha, Ritam, et al. Moaz: A multi-objective automl-zero framework. Proceedings of the Genetic and Evolutionary Computation Conference. 2023.
- [3] So, D. R., Liang, C., and Le, Q. V. The evolved transformer. In ICML, 2019.
- [4] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and TAMT Meyarivan. A fast and elitist multiobjective genetic algorithm: Nsga-ii. IEEE Transactions on Evolutionary Computation, 6(2):182–197, 2002.
- [5] Lange, Robert Tjarko, Yuki Imajuku, and Edoardo Cetin. Shinkaevolve: Towards open-ended and sample-efficient program evolution. arXiv preprint arXiv:2509.19349 (2025).
- [6] AlexanderNovikov, NganVu, MarvinEisenberger, EmilienDupont, Po-SenHuang, AdamZsoltWagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. arXiv preprint arXiv:2506.13131, 2025.
- [7] Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. Machine learning, 47(2):235–256, 2002.
- [8] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. arXiv preprint arXiv:2408.08435, 2024.
- [9] Yuki Imajuku, Kohki Horie, Yoichi Iwata, Kensho Aoki, Naohiro Takahashi, and Takuya Akiba. Ale-bench: A benchmark for long-horizon objective-driven algorithm engineering. arXiv preprint arXiv:2506.09050, 2025.
- [10] He, Xin, Kaiyong Zhao, and Xiaowen Chu. AutoML: A survey of the state-of-the-art. Knowledge-based systems 212 (2021): 106622.
- [11] B. Zoph, Q. V. Le, Neural architecture search with reinforcement learning, in: 5th International Conference on Learning Representations, ICLR

- 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings, OpenReview.net, 2017.
- [12] I. F. Ilyas, Effective data cleaning with continuous evaluation., IEEE Data Eng. Bull. 39 (2) (2016) 38–46.
 - [13] M. Mahdavi, F. Neutatz, L. Visengeriyeva, Z. Abedjan, Towards automated data cleaning workflows, Machine Learning 15 (2019) 16.
 - [14] T. Wei, C. Wang, Y. Rui, C. W. Chen, Network morphism, in: M. Balcan, K. Q. Weinberger (Eds.), Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016, Vol. 48 of JMLR Workshop and Conference Proceedings, JMLR.org, 2016, pp. 564–572.
 - [15] B. Zoph, Q. V. Le, Neural architecture search with reinforcement learning, in: 5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings, OpenReview.net, 2017. URL <https://openreview.net/forum?id=r1Ue8Hcxg>
 - [16] Nikolaus Hansen, Dimo Brockhoff, Olaf Mersmann, Tea Tusar, Dejan Tusar, Ouassim Ait ElHara, Phillipe R. Sampaio, Asma Atamna, Konstantinos Varelas, Umut Batu, Duc Manh Nguyen, Filip Matzner, Anne Auger. COMparing Continuous Optimizers: numbbbo/COCO on Github. Zenodo, DOI:10.5281/zenodo.2594848, March 2019.
 - [17] de Rosa, Gustavo H., Douglas Rodrigues, and João P. Papa. "Opytimizer: A nature-inspired python optimizer."arXiv preprint arXiv:1912.13002 (2019).
 - [18] Finck, Steffen, et al. Real-parameter black-box optimization benchmarking 2009: Presentation of the noiseless functions. Technical Report 2009/20, Research Center PPE, 2009. Updated February, 2010.